

Integrations & No-Code Tool Matrix

1. Recommended Production Architecture & Justification

Architecture Overview

- **Frontend Application:** WeWeb (Native SPA, high-density CSS grids, Chart.js and Mapbox GL embeds, direct REST/GraphQL connectors, no vendor lock-in).
- **Database and Auth:** Supabase (Managed PostgreSQL 15+, GoTrue RBAC JWT auth, native Row-Level Security, Edge Functions).
- **Analytics Read-Replica:** Dedicated PostgreSQL replica handling heavy cross-scheme analytical joins and PostGIS spatial queries.
- **Automation and ETL:** n8n (Self-hosted on Railway or AWS, zero per-run fees, sovereign data residency, high-volume batch processing).
- **AI Reasoning Layer:** Python FastAPI microservice with Azure OpenAI / Anthropic via LangChain for Text-to-SQL generation and AST verification.

Core Stack Trade-Off & Evaluation Matrix

Architectural Layer	Selected Tool	Evaluated Alternatives	Primary Technical & Operational Justifications
Frontend Framework	WeWeb	Bubble, Retool	Compiles to native Vue.js/Vite SPAs; supports direct connections to external PostgreSQL/Supabase with native RLS; eliminates vendor lock-in via raw code exports.
Database & Auth	Supabase	Firebase, Airtable	Provides full enterprise-grade PostgreSQL with PostGIS extension; native Row-Level Security (RLS); supports complex multi-table analytical queries without record limits.
ETL & Data Pipeline	n8n	Zapier, Make	Zero per-execution operational costs when self-hosted; strict data sovereignty compliance;

Architectural Layer	Selected Tool	Evaluated Alternatives	Primary Technical & Operational Justifications
			raw payload streaming without restrictive size limitations.
AI Orchestration	FastAPI / LangChain	Direct API Call	Custom AST SQL verification layer; strict context injection; secure role-based prompt filtering prior to database execution.

Stack Justifications

Why WeWeb over Bubble

Bubble utilizes a proprietary backend that struggles with complex SQL joins across millions of records and does not support native PostgreSQL Row-Level Security. WeWeb compiles to standard Vue.js/Vite Single Page Applications, connects directly to external databases via standard REST/GraphQL APIs, and natively integrates with Supabase authentication and RLS.

Why Supabase over Firebase/Airtable

Airtable hard-caps base records (typically 50k to 125k), failing the volume requirements of national governance datasets. Firebase (NoSQL Firestore) cannot natively perform multi-table relational joins, geographic entity lookups, or enforce complex SQL-level RLS policies. Supabase provides enterprise-grade PostgreSQL handling relational depth, JSONB payloads, and spatial queries via PostGIS.

Why n8n over Zapier/Make

National government datasets contain millions of rows requiring heavy batch ingestion. Make and Zapier charge per-task fees that become cost-prohibitive at scale and impose payload size limits (typically 5MB to 10MB). Self-hosted n8n operates with zero per-run execution costs, supports deployment within sovereign cloud boundaries, and handles large data streaming pipelines with full control over timeout and retry policies.

2. Third-Party APIs & Integration Specifications

Integration 1: Open Government Data (data.gov.in) / Scheme Endpoints

- **Purpose:** Daily sync of scheme physical and financial implementation files.
- **Auth Method:** API-Key passed via headers (X-OGD-API-Key: <TOKEN>).

- **Endpoint Pattern:** GET

```
api.data.gov.in/resource/{resource_id}?api-key={key}&format=json&limit=1000&offset={offset}
```

Request Headers

- **Accept:** application/json
- **X-OGD-API-Key:** d98a2f389cbe44b58e76a521

Sample Ingestion Response Payload

```
{  
  
  "status": "ok",  
  
  "total": 35200,  
  
  "records": [  
  
    {  
  
      "state_name": "Uttar Pradesh",  
  
      "district_name": "Gorakhpur",  
  
      "scheme_code": "PMAY_G",  
  
      "financial_year": "2025-2026",  
  
      "houses_sanctioned": 14250,  
  
      "houses_completed": 9820,  
  
      "funds_released_lakhs": 11784.00,  
  
      "funds_utilized_lakhs": 9812.50,  
  
      "last_updated": "2026-03-25T14:30:00Z"  
  
    }  
  
  ]  
  
}
```

Integration 2: AI Reasoning & Text-to-SQL Orchestrator

- **Purpose:** Converts conversational natural-language governance questions into secure, traceable SQL and evidence-backed summaries.
- **Auth Method:** Bearer token (`Authorization: Bearer <SUPABASE_SERVICE_ROLE_KEY>`).
- **Endpoint Pattern:** `POST internal-orchestrator.domain/v1/query`

Request Payload

```
{  
  
  "user_id": "usr_8f7b2c14-5d3e-4b7a-9a1c-3e2f5b6a7c8d",  
  
  "user_role": "central_executive",  
  
  "assigned_lgd": null,  
  
  "query": "Which districts in Bihar show fund utilization under 50 percent for MGNREGA despite high demand?",  
  
  "execution_context": {  
  
    "fiscal_year": "2025-2026",  
  
    "enforce_limit": 100  
  
  }  
  
}
```

Response Payload (with Data Provenance and Citations)

```
{  
  
  "query_id": "qry_99a8b7c6-1d2e-3f4a-5b6c-7d8e9f0a1b2c",  
  
  "execution_time_ms": 142,  
  
  "synthesized_response": "Identified 3 districts in Bihar (Araria, Kishanganj, Purnia) exhibiting severe fund utilization lags (<50%) despite generating over 2.5 million person-days of labor demand in FY 2025-26.",  
  
  "generated_sql": "SELECT g.entity_name AS district_name, a.allocated_cr, a.utilized_cr, (a.utilized_cr / NULLIF(a.allocated_cr, 0) * 100) AS utilization_rate, b.achieved_units AS person_days FROM fact_scheme_allocations a JOIN dim_lgd_geography g ON a.lgd_code ="
```

```
g.lgd_code JOIN fact_scheme_beneficiaries_agg b ON a.scheme_id = b.scheme_id AND
a.lgd_code = b.lgd_code WHERE g.state_lgd = 10 AND a.scheme_id =
'c1234567-0000-0000-0000-000000000001' AND a.financial_year = '2025-2026' AND
(a.utilized_cr / NULLIF(a.allocated_cr, 0)) < 0.50 ORDER BY utilization_rate ASC LIMIT 100;
```

```
"evidence_provenance": {

  "source_tables": [

    "fact_scheme_allocations",

    "fact_scheme_beneficiaries_agg",

    "dim_lgd_geography"

  ],

  "data_freshness": "2026-03-31T20:30:00Z",

  "citations": [

    {

      "district_lgd": 216,

      "district_name": "Araria",

      "utilization_rate": "38.2%",

      "source_id": "alloc_row_9812"

    },

    {

      "district_lgd": 222,

      "district_name": "Kishanganj",

      "utilization_rate": "41.6%",

      "source_id": "alloc_row_9815"

    }

  ]

}
```

```

]

},

"visualization_recommendation": {

  "type": "bar_horizontal",

  "x_axis": "utilization_rate",

  "y_axis": "district_name"

}

}

```

Integration 3: GIS Vector Tile Server (NIC / Mapbox Services)

- **Purpose:** Renders interactive, pan-and-zoom India administrative boundaries down to district and block levels.
- **Auth Method:** URL Query Token or Access Token (`access_token=<MAP_TOKEN>`).
- **Endpoint Pattern:** `GET`
`gis-tiles.internal.domain/v1/tiles/lgd/{z}/{x}/{y}.pbf?layer=district`
`&year=2026`
- **Response:** Protocol Buffer Vector Tile (`.pbf`) dynamically parsed by Mapbox GL / MapLibre JS running inside the WeWeb custom application frame. Boundaries bind directly to `dim_lgd_geography.lgd_code` feature properties to enable real-time thematic color fills based on Postgres query outputs.